

Comparing CYK Parsing Algorithms

Abdullah Saeed

abdullah391261@gmail.com

Abstract.

1 Introduction

2 Background

2.1 Context Free Grammars and the Membership Problem

A context free grammar (CFG) describes a language using a set of production rules. A CFG can be defined as

$$G = V, \Sigma, R, S$$

where V is a set of nonterminals, Σ is a finite set of terminals, R is a set of production rules, and $S \in V$ is the start symbol. The production rules describe how the nonterminals can be replaced.

Starting from S , these rules can be applied to generate strings of terminal symbols. The set of strings that can be generated by the grammar is called the language of the grammar. The membership problem is to determine whether a given input string belongs to this language.

2.2 Chomsky Normal Form

The parsing algorithms considered in this report use grammars in Chomsky normal form (CNF). The production rules used have one of the following two forms:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a,$$

where A , B , and C are nonterminals and a is a terminal.

2.3 CYK and Dynamic Programming

The Cocke Younger Kasami (CYK) algorithm is used to check if an input string belongs to the language of a grammar. Instead of looking at the whole string at once, CYK works with smaller parts of the string. We can describe one of these problems as (A, i, j) , where we want to know if the nonterminal A can generate the part of the string between positions i and j . If the substring has only one terminal, we only need to check if there is a terminal rule for it. For longer substrings, the algorithm tries rules of the form $A \rightarrow BC$ and different positions where the substring can be split.

Dynamic programming is useful here because the same smaller problem may be needed more than once. In bottom-up CYK, we first solve the problems for short substrings and then use these results when working with longer ones. The top-down version works in the other direction. It starts with the full input string and recursively checks the smaller problems that are needed. Memoization is used to remember results that have already been found, so they do not need to be calculated again [1]. The naive recursive version does not use memoization, which means that it may solve the same problem many times.

3 Algorithms

Three different parsing algorithms are used in this work: naive recursive, bottom-up CYK, and top-down CYK with memoization. All three are used to solve the same membership problem, but they handle the smaller subproblems in different ways.

3.1 Naive

The naive algorithm uses recursion and does not save any previous results. Parsing starts with the start symbol and the complete input string. A call $\text{PARSE}(A, i, j)$ checks if A can generate the part of the input between positions i and j .

When the substring has length one, the algorithm checks if there is a terminal rule that matches the input symbol. For longer substrings, it goes through the rules $A \rightarrow BC$ and tries the possible split positions k . The left and right parts are then checked recursively. If both parts can be generated, the algorithm returns true. Otherwise it continues trying the remaining possibilities.

```

Parse(A, i, j)
  if j = i + 1
    return whether A -> input[i] is a rule
  for each rule A -> B C
    for k = i + 1 to j - 1
      if Parse(B, i, k)
        if Parse(C, k, j)
          return True
  return False

```

Parsing the complete input is done with $\text{PARSE}(S, 0, n)$, where S is the start symbol and n is the input length.

3.2 Bottom-Up

The bottom-up algorithm uses a table instead of recursion. It first fills the table for substrings of length one by checking the terminal rules. After this, it moves to longer substrings until it reaches the complete input string.

For each substring, the algorithm checks the binary rules $A \rightarrow BC$ and the different places where the substring can be split. If the table already shows that B can generate the left part and C can generate the right part, the entry for A is set to true.

```

BottomUp(input)
  create table and initialize all entries to False
  for each nonterminal A
    for each position i
      if A -> input[i] is a rule
        table[A, i, i+1] = True
  for length = 2 to n
    for each substring (i, j) of this length
      for each nonterminal A
        for each rule A -> B C
          for k = i + 1 to j - 1
            if table[B, i, k] and table[C, k, j]
              table[A, i, j] = True
              break
  return table[S, 0, n]

```

The final result is found in $\text{table}[S, 0, n]$, which tells if the start symbol can generate the complete input.

3.3 Top-Down

The top-down algorithm is similar to the naive recursive algorithm, but it stores results that have already been calculated. Before solving a subproblem, it first checks the table. If the result is already there, it can be returned directly instead of doing the same work again.

If the result is not in the table, the algorithm continues in the same way as the naive version. It checks the terminal rule for a substring of length one, or tries the binary rules and split positions for a longer substring. Before returning the result, it is saved in the table.

```

Parse(A, i, j)
  if table[A, i, j] has stored a result
    return table[A, i, j]
  if j = i + 1
    result = whether A -> input[i] is a rule
    table[A, i, j] = result

```

```
    return result
  for each rule A -> B C
    for k = i + 1 to j - 1
      if Parse(B, i, k)
        if Parse(C, k, j)
          table[A, i, j] = True
          return True
    table[A, i, j] = False
  return False
```

The algorithm starts with $\text{PARSE}(S, 0, n)$. The difference from the naive version is that previously calculated results can now be reused.

4 Theoretical Analysis

The three algorithms solve the same membership problem, but they can perform very different amounts of work. In the following analysis, the grammar is considered fixed and n is the length of the input string.

4.1 Naive

The naive algorithm does not store the results of previous subproblems. The same problem (A, i, j) can therefore be reached through different recursive calls and calculated again each time [1]. As the input becomes longer, the recursive search can branch into many possibilities, and some of the same subproblems may occur in several of these branches. Because of this repeated work, the number of recursive calls can grow exponentially in the worst case.

This can happen because one recursive call may lead to several new recursive calls for different rules and split positions, and the same subproblems can then be reached again through different branches.

This does not mean that every input requires exponential work. The algorithm returns as soon as it finds a rule and split that successfully generate the substring. If a successful derivation is found early, only a small part of the possible recursive search may be needed. The running time can therefore depend strongly on the grammar and the input string.

4.2 Bottom-Up

The bottom-up algorithm fills the table for substrings of increasing length. For an input of length n , there are at most n different substring lengths and at most n possible starting positions. For each substring, the algorithm may also have to try up to n different split positions.

$$O(n) \cdot O(n) \cdot O(n) = O(n^3)$$

This gives $O(n^3)$ time in the worst case. The table stores results for pairs of input positions (i, j) . There are $O(n^2)$ such pairs. Therefore the space complexity is $O(n^2)$.

Bottom-up fills the table step by step, so it may calculate entries that are not needed for the final result.

4.3 Top-Down

The top-down algorithm uses the same recursive idea as the naive algorithm, but memoization prevents a subproblem from being completely calculated more than once [1]. A subproblem is identified by (A, i, j) . For a fixed grammar, there are $O(n^2)$ possible pairs of positions (i, j) and therefore $O(n^2)$ different subproblems.

When a new subproblem is calculated, the algorithm may try up to $O(n)$ split positions. The worst-case running time is therefore

$$O(n^2) \cdot O(n) = O(n^3).$$

The memoization table can contain $O(n^2)$ stored results, giving a space complexity of $O(n^2)$.

Although top-down and bottom-up have the same worst case time complexity, they do not necessarily perform the same amount of work on every input. Bottom-up fills its table systematically, while top-down only calculates subproblems reached by the recursive search. The naive algorithm can also perform little work when a successful derivation is found early, but it can repeat the same calculations many times on less favourable inputs. The differences between the algorithms can therefore depend on both the grammar and the structure of the input string.

5 Experiments

The experiments compare the three algorithms using both operation counts and execution time. For the naive and top-down algorithms, the operation counter is increased for each recursive call. For bottom-up, it is increased for each execution of the innermost loop. The counter gives an estimate of the work performed without depending on the execution time.

For the time measurements, each input was run ten times. The highest and lowest times were removed and the average of the remaining runs was used. A warm-up run with the longest input was performed before the measurements.

5.1 Balanced Parentheses

The first experiment used the following grammar:

$$\begin{aligned} S &\rightarrow SS \mid T_l S_p \mid T_l T_r, \\ S_p &\rightarrow S T_r, \\ T_l &\rightarrow (, \\ T_r &\rightarrow). \end{aligned}$$

Two types of valid input strings were tested. The first consists of repeated pairs of parentheses $()() \dots$, while the second used nested parentheses $((\dots))$

Input lengths from 4 to 12 were used. The grammar and input lengths remained the same, while the structure of the strings differed. This allows us to compare how the structure of a valid input affects the work performed by the three algorithms.

5.2 Position of the Terminal a

The second experiment uses the grammar:

$$\begin{aligned} A &\rightarrow AB \mid AA \mid a, \\ B &\rightarrow BB \mid b, \end{aligned}$$

where A is the start symbol. This grammar generates strings over a and b that start with an a .

The two forms tested were $ab \dots b$ and $b \dots ba$.

The input lengths were 5, 7, 9, 11, and 13. For each length, both inputs contain one a and the same number of b symbols, but the position of the a is different. The first form belongs to the language of the grammar, while the second does not. This experiment is used to examine how a small change in the arrangement of the input can affect the search performed by the algorithms.

5.3 Running Time Growth

The third experiment focuses on the running time of the two CYK algorithms. The purpose is to compare the measured running times with the theoretical $O(n^3)$ running time discussed in Section 4.

The grammar used is

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow AA \mid a, \\ B &\rightarrow BB \mid b. \end{aligned}$$

The input strings have the form $a \dots ab$ and the lengths from 20 to 200 were tested. Only the top-down and bottom-up algorithms are used in this experiment. Larger inputs are useful here because the running times are easier to compare as the input length increases, while the naive algorithm can become too expensive on larger inputs.

The running time is measured using the same procedure as in the other experiments. This experiment is used to see whether the growth in measured running time is consistent with the cubic running-time analysis of the two CYK algorithms.

6 Results

This section presents the results of the experiments described in the previous section.

6.1 Balanced Parentheses

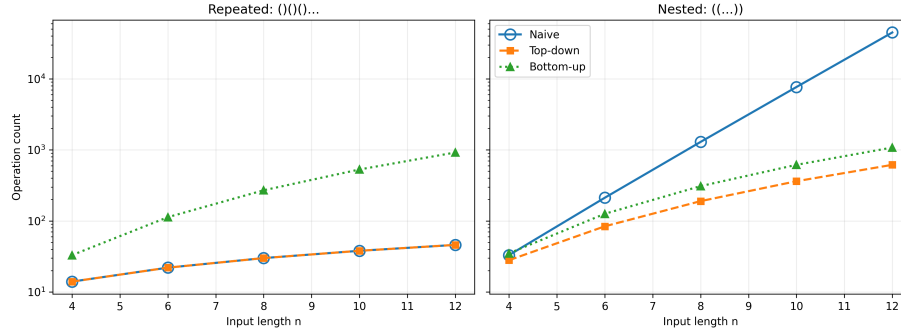


Fig. 1. Operation counts for repeated and nested balanced parentheses inputs.

For repeated parentheses, naive and top-down perform the same number of operations for all tested lengths. At length 12, both perform 46 operations, compared with 919 for bottom-up.

For nested parentheses, the results are very different. At length 12, naive performs 44,877 operations, while top-down performs 616 and bottom-up performs 1,079. The corresponding running times were approximately 6.78 ms for naive, 0.091 ms for top-down, and 0.108 ms for bottom-up.

6.2 Position of the Terminal a

For $ab \dots b$, naive and top-down have the same operation counts. At length 13, both perform 25 operations, while bottom-up performs 728.

For $b \dots ba$, the difference is much larger. At length 13, naive performs 531,441 operations, compared with 157 for top-down and 872 for bottom-up. The running times for the same input were approximately 90.39 ms, 0.023 ms, and 0.101 ms respectively.

6.3 Running Time Growth

For top-down, the running time increases from approximately 0.00022 seconds at length 20 to 0.153 seconds at length 200. Bottom-up increases from approximately 0.00031 seconds to 0.169 seconds over the same range. The two curves show similar growth, with top-down being slightly faster throughout this experiment.

The observed growth is consistent with the cubic running-time analysis from Section 4.

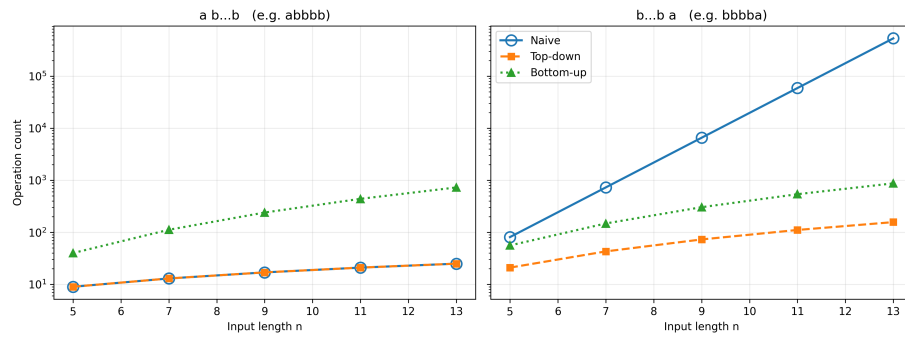


Fig. 2. Operation counts for inputs of the forms $ab \dots b$ and $b \dots ba$.

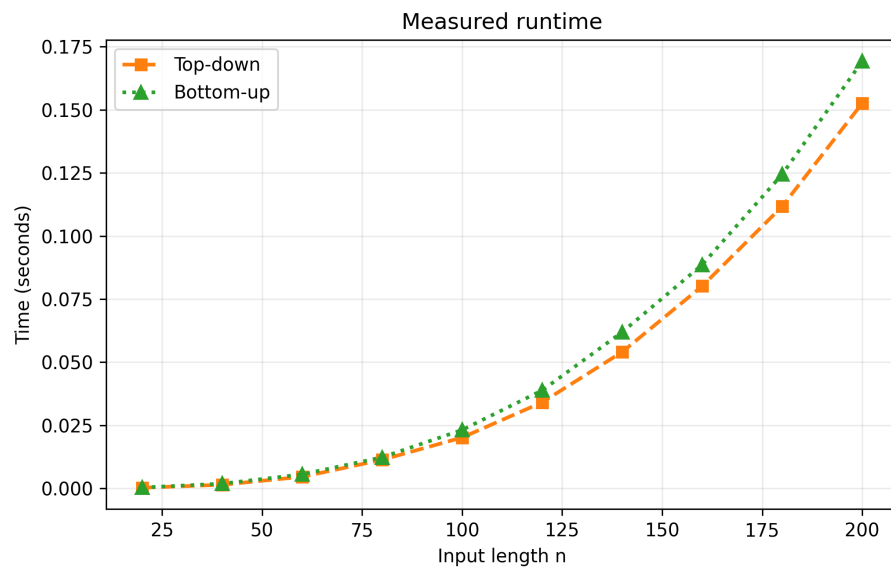


Fig. 3. Running times of the top-down and bottom-up algorithms for increasing input lengths.

7 Discussion

The experiments show that the performance of the algorithms depends not only on the input length, but also on the structure of the input and how the parser reaches a successful or unsuccessful result.

For repeated parentheses, the naive and top-down algorithms performed the same amount of counted work. In this case, the recursive search can find a successful derivation without repeatedly solving many of the same subproblems. Memoization therefore gives little benefit. Bottom-up still fills its table systematically, including entries that are not needed for the final result.

The nested inputs behave differently. The naive algorithm reaches the same subproblems through different recursive branches and calculates them again each time. Top-down can avoid much of this repeated work because previously calculated results are stored in the memoization table. This explains the much larger difference between naive and top-down for the nested inputs.

The second experiment shows a similar effect more clearly. For inputs of the form $ab \dots b$, the recursive algorithms find a successful derivation early. When the a is moved to the end, the input is not in the language of the grammar. The recursive search therefore cannot stop after finding an early successful derivation and has to explore many more possibilities. This affects the naive algorithm especially strongly because results from repeated subproblems are not stored. Top-down also has to explore more of the search, but memoization prevents the same subproblems from being fully calculated again.

These results also show why the worst-case complexity does not describe the running time of every individual input. The naive algorithm can perform very little work on favourable inputs even though its worst-case behaviour is much worse. Similarly, top-down and bottom-up have the same cubic worst-case bound, but they do not necessarily calculate the same subproblems. Top-down only calculates entries reached by its recursive search, while bottom-up works through the table systematically.

The physical running times generally follow the same pattern as the operation counts, especially when the differences in operation counts are large. For small and favourable inputs, however, naive can be faster than top-down even when their operation counts are the same. In these cases, memoization does not save work but still requires checking and storing table entries. The operation counter should therefore be viewed as an estimate of algorithmic work rather than an exact measure of execution time.

Finally, the running-time experiment with larger inputs showed similar growth for the top-down and bottom-up algorithms. This agrees with the cubic worst case analysis, while the difference between their measured times shows that having the same asymptotic complexity does not mean having exactly the same running time in practice.

References

1. Kleinberg, J., Tardos, E.: Algorithm Design. Pearson Education (2006)